## المذكرة الاحترافية

**Software Engineering for Machine Learning Engineers**

# الفصل الخامس: Project Structure

*ترجمة مبادئ Clean Architecture لتنظيم فعلي وملموس على مستوى المجلدات والملفات*

Modular Design • Packages • Configuration • Environment Variables

# جدول المحتويات


جدول المحتويات ..... 2
مقدمة الفصل الخامس: من المعمارية النظرية إلى الملفات الفعلية ..... 5
٥٫١ Modular Design — التصميم المعياري ..... 6
١. ليه التصميم المعياري مهم؟ ..... 6
٢. المشكلة اللي بيحلها التصميم المعياري ..... 6
٣. إزاي كانت الشركات بتتعامل قبل تبني التصميم المعياري ..... 6
٤. ليه الطريقة القديمة مش مناسبة ..... 6
٥. الفكرة الأساسية وراء التصميم المعياري ..... 6
٦. المفاهيم الأساسية ..... 6
٧. المكونات: أنواع الوحدات الشائعة في مشروع ML ..... 6
٨. إزاي المبدأ ده بيشتغل داخليًا ..... 6
٩. إزاي بيتم استخدامه داخل الشركات الكبرى ..... 7
١٠. فين موقعه داخل مشروع ML حقيقي ..... 7
١١. امتى تلتزم بتصميم معياري صارم ..... 7
١٢. امتى ممكن تتساهل شوية ..... 7
١٣. أفضل الممارسات ..... 7
١٤. أشهر الأخطاء الشائعة ..... 7
١٥. اعتبارات الأداء ..... 7
١٦. الاعتبارات الأمنية ..... 7
١٧. اعتبارات قابلية التوسع ..... 8
١٨. مقارنة مع نهج بديل ..... 8
١٩. مثال عملي صغير ..... 8
٢٠. مثال من مشروع Production حقيقي ..... 8
٢١. ملخص الموضوع ..... 8
٥٫٢ Packages — الحزم البرمجية ..... 9
١. ليه مفهوم الحزم مهم؟ ..... 9
٢. المشكلة اللي بتحلها الحزم ..... 9
٣. إزاي كانت الشركات بتتعامل قبل هيكلة الحزم بشكل احترافي ..... 9
٤. ليه الطريقة القديمة مش مناسبة ..... 9
٥. الفكرة الأساسية وراء الحزم ..... 9
٦. المفاهيم الأساسية ..... 9
٧. المكونات ..... 9
٨. إزاي المبدأ ده بيشتغل داخليًا ..... 9
٩. إزاي بيتم استخدامه داخل الشركات الكبرى ..... 9
١٠. فين موقعه داخل مشروع ML حقيقي ..... 10
١١. امتى تنظم الكود كحزمة رسمية ..... 10
١٢. امتى ممكن تتجنب الهيكلة الرسمية للحزم ..... 10
١٣. أفضل الممارسات ..... 10
١٤. أشهر الأخطاء الشائعة ..... 10
١٥. اعتبارات الأداء ..... 10

١٦. الاعتبارات الأمنية ..... 10
١٧. اعتبارات قابلية التوسع ..... 10
١٨. مقارنة مع نهج بديل ..... 11
١٩. مثال عملي صغير ..... 11
٢٠. مثال من مشروع Production حقيقي ..... 11
٢١. ملخص الموضوع ..... 11
٥٫٣ Configuration — إدارة الإعدادات ..... 12
١. ليه إدارة الإعدادات بشكل صحيح مهمة؟ ..... 12
٢. المشكلة اللي بتحلها إدارة الإعدادات الصحيحة ..... 12
٣. إزاي كانت الشركات بتتعامل قبل إدارة الإعدادات بشكل منظم ..... 12
٤. ليه الطريقة القديمة مش مناسبة ..... 12
٥. الفكرة الأساسية وراء إدارة الإعدادات ..... 12
٦. المفاهيم الأساسية ..... 12
٧. المكونات ..... 12
٨. إزاي المبدأ ده بيشتغل داخليًا ..... 12
٩. إزاي بيتم استخدامه داخل الشركات الكبرى ..... 13
١٠. فين موقعه داخل مشروع ML حقيقي ..... 13
١١. امتى تفصل الإعدادات عن الكود بصرامة ..... 13
١٢. امتى ممكن تسيب القيمة جوه الكود ..... 13
١٣. أفضل الممارسات ..... 13
١٤. أشهر الأخطاء الشائعة ..... 13
١٥. اعتبارات الأداء ..... 13
١٦. الاعتبارات الأمنية ..... 13
١٧. اعتبارات قابلية التوسع ..... 14
١٨. مقارنة مع نهج بديل ..... 14
١٩. مثال عملي صغير ..... 14
٢٠. مثال من مشروع Production حقيقي ..... 14
٢١. ملخص الموضوع ..... 14
٥٫٤ Environment Variables — متغيرات البيئة ..... 16
١. ليه متغيرات البيئة مهمة؟ ..... 16
٢. المشكلة اللي بتحلها متغيرات البيئة ..... 16
٣. إزاي كانت الشركات بتتعامل قبل استخدام متغيرات البيئة بشكل صحيح ..... 16
٤. ليه الطريقة القديمة مش مناسبة ..... 16
٥. الفكرة الأساسية وراء متغيرات البيئة ..... 16
٦. المفاهيم الأساسية ..... 16
٧. المكونات ..... 16
٨. إزاي المبدأ ده بيشتغل داخليًا ..... 16
٩. إزاي بيتم استخدامه داخل الشركات الكبرى ..... 17
١٠. فين موقعه داخل مشروع ML حقيقي ..... 17
١١. امتى تستخدم متغيرات البيئة ..... 17
١٢. امتى ملفات الإعدادات العادية كافية ..... 17

١٣. أفضل الممارسات ..... 17
١٤. أشهر الأخطاء الشائعة ..... 17
١٥. اعتبارات الأداء ..... 17
١٦. الاعتبارات الأمنية ..... 17
١٧. اعتبارات قابلية التوسع ..... 18
١٨. مقارنة مع نهج بديل ..... 18
١٩. مثال عملي صغير ..... 18
٢٠. مثال من مشروع Production حقيقي ..... 18
٢١. ملخص الموضوع ..... 18
مشروع عملي على الفصل: إعادة تنظيم مشروع ML من سكربت واحد لبنية احترافية كاملة ..... 19
خطوات التنفيذ ..... 19
تحديات شائعة عند التطبيق ..... 19
إزاي الفصل ده بيتستخدم داخل مشروع Machine Learning حقيقي ..... 19
ملخص الفصل الخامس وربطه بالفصل القادم ..... 19

## مقدمة الفصل الخامس: من المعمارية النظرية إلى الملفات الفعلية

في الفصل اللي فات اتعلمنا Clean Architecture: إزاي ننظم النظام في طبقات (Layers)، وإزاي نتحكم في اتجاه الاعتماد بين الطبقات دي (Dependency Rule)، وإزاي فصل الاهتمامات (Separation of Concerns) هو المبدأ الأشمل اللي بيربط كل ده مع بعض. لكن كل ده كان على مستوى نظري ومعماري، من غير ما نشوف إزاي المفاهيم دي بتترجم فعليًا لمجلدات وملفات حقيقية على القرص.

الفصل ده هيسد الفجوة دي: Project Structure هو التطبيق الملموس والعملي لكل مبادئ Clean Architecture اللي اتعلمناها، على مستوى تنظيم المجلدات (Modular Design وPackages)، وعلى مستوى إدارة الإعدادات اللي بتحدد سلوك النظام في بيئات مختلفة (Configuration وEnvironment Variables).

ليه بندرس الموضوع ده دلوقتي؟ لأن مهما كانت بنيتك المعمارية صحيحة نظريًا، لو مترجمتش لتنظيم واضح وواقعي للمجلدات والملفات، أي مهندس جديد هيلاقي صعوبة يعرف فين يحط كود جديد أو فين يدور على كود موجود. التنظيم الفعلي للمشروع هو أول انطباع بيكونه أي مهندس عن جودة المشروع كله.

وهنحتاج الموضوع ده بشكل مباشر في الفصل الجاي (Error Handling)، لأن التعامل مع الأخطاء والاستثناءات بشكل احترافي بيعتمد على وجود تنظيم واضح يحدد فين المفروض يتم التعامل مع كل نوع خطأ، وإزاي الإعدادات المختلفة (زي مستوى التفاصيل في الـ Logging) بتتغير حسب البيئة اللي النظام شغال فيها.

# ٥,١ Modular Design — التصميم المعياري

*التصميم المعياري هو تقسيم المشروع لوحدات (Modules) مستقلة، كل وحدة بتمثل جزء واضح من وظيفة النظام، وده الترجمة العملية لفكرة الطبقات وفصل الاهتمامات اللي اتعلمناها في الفصل اللي فات على مستوى تنظيم المجلدات الفعلي.*

## ١. ليه التصميم المعياري مهم؟

مشروع من غير تصميم معياري واضح بيتحول لمجموعة ملفات عشوائية، صعب على أي مهندس (حتى لو هو نفسه اللي كتب الكود) يلاقي مكان أي جزء معين بسرعة. التصميم المعياري بيوفر خريطة واضحة: كل وحدة ليها مكان محدد ومسؤولية واضحة.

في مشاريع ML، التصميم المعياري بيسمح لفرق مختلفة (بيانات، نمذجة، نشر) يشتغلوا على وحدات منفصلة من نفس المشروع من غير تعارض مستمر.

## ٢. المشكلة اللي بيحلها التصميم المعياري

المشكلة هي المشاريع اللي كل الكود فيها موجود في ملف واحد ضخم أو مجموعة ملفات من غير تنظيم منطقي، حيث كود تحميل البيانات، ومنطق النموذج، وكود التقييم كلهم مبعثرين في نفس المكان من غير حدود واضحة بينهم.

## ٣. إزاي كانت الشركات بتتعامل قبل تبني التصميم المعياري

في المشاريع المبكرة والنماذج الأولية، كان شائع كتابة كل الكود في سكربت واحد أو Notebook واحد، خصوصًا في مرحلة الاستكشاف السريع للفكرة، من غير التفكير في تقسيم منطقي للمشروع.

## ٤. ليه الطريقة القديمة مش مناسبة

مع نمو المشروع، الملف الواحد الضخم بيصبح صعب التنقل فيه، وأي مهندس جديد بيحتاج وقت طويل عشان يفهم فين الجزء اللي محتاج يعدله، وده بيبطئ سرعة الفريق بشكل عام.

## ٥. الفكرة الأساسية وراء التصميم المعياري

المشروع بيتقسم لوحدات (Modules) أو حزم (Packages) مستقلة، كل واحدة بتمثل مسؤولية واضحة (زي data، models، evaluation، training)، وكل وحدة بتحتوي على الكود المرتبط بمسؤوليتها بس، مع حدود واضحة (Interfaces) للتواصل مع الوحدات التانية.

## ٦. المفاهيم الأساسية

- Module: وحدة كود مستقلة (عادة ملف أو مجلد) بتمثل مسؤولية واضحة ومحددة.
- High Cohesion: العناصر جوه الوحدة الواحدة مرتبطة منطقيًا ببعض، بنفس فكرة الترابط اللي اتعلمناها في فصل Classes.
- Low Coupling: الوحدات المختلفة مرتبطة ببعض بأقل قدر ممكن، وأي تواصل بينهم بيتم عن طريق واجهات واضحة.

## ٧. المكونات: أنواع الوحدات الشائعة في مشروع ML

- وحدة data: مسؤولة عن تحميل ومعالجة البيانات.
- وحدة models: مسؤولة عن تعريف بنية النماذج المختلفة.
- وحدة training: مسؤولة عن منطق التدريب وحلقة التدريب.
- وحدة evaluation: مسؤولة عن حساب المقاييس وتقييم الأداء.
- وحدة utils أو common: مسؤولة عن أدوات مشتركة مستخدمة عبر الوحدات التانية.

## ٨. إزاي المبدأ ده بيشتغل داخليًا

لما كل وحدة معزولة بمسؤولية واضحة، أي تغيير في وحدة معينة (زي تحسين طريقة تحميل البيانات) بيأثر بس على الكود اللي فعلًا بيستخدم الوحدة دي، وده بيقلل نطاق التأثير لأي تعديل ويسهل فهم النظام جزء جزء بدل الحاجة لاستيعابه بالكامل مرة واحدة.

### ٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في منصات ML الكبرى، بيتم تقسيم المشروع لحزم (Packages) منفصلة بوضوح، أحيانًا كل حزمة بتكون قابلة للتوزيع (Installable Package) بشكل مستقل، بحيث فرق مختلفة تقدر تستورد وتستخدم وحدات معينة (زي وحدة معالجة البيانات المشتركة) من غير الحاجة لنسخ الكود أو الاعتماد على المشروع كله.

### ١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في التنظيم الأساسي لأي مشروع ML احترافي: مجلد منفصل لكل مسؤولية رئيسية، وملفات __init__.py بتحدد الواجهة العامة لكل وحدة، وفصل واضح بين الكود القابل لإعادة الاستخدام (Library Code) وكود التشغيل الفعلي (Scripts أو Entry Points).

### ١١. امتى تلتزم بتصميم معياري صارم

- في أي مشروع متوقع يكبر ويتطور لفترة طويلة.
- لما فرق مختلفة متوقع تشتغل على أجزاء مختلفة من نفس المشروع.

### ١٢. امتى ممكن تتساهل شوية

- في سكربت تجريبي صغير جدًا أو Notebook استكشافي شخصي مش متوقع يتطور لمشروع كبير.
- لكن لازم خطة واضحة لإعادة التنظيم لو الفكرة نجحت وهتدخل مرحلة التطوير الفعلي.

### ١٣. أفضل الممارسات

**Best Practices**

- قسم المشروع لوحدات حسب المسؤولية الوظيفية، مش حسب نوع الملف (زي كل الكلاسات في ملف واحد بغض النظر عن وظيفتها).
- خلي كل وحدة عالية الترابط داخليًا ومنخفضة الاعتماد على الوحدات التانية.
- استخدم ملفات __init__.py لتحديد الواجهة العامة اللي كل وحدة بتعرضها للوحدات التانية.

### ١٤. أشهر الأخطاء الشائعة

**Common Mistakes**

- تقسيم الوحدات حسب نوع تقني (زي مجلد classes ومجلد functions) بدل حسب المسؤولية الوظيفية الفعلية.
- خلق اعتماد دائري (Circular Dependency) بين وحدات بتحتاج بعض في نفس الوقت.
- وحدة واحدة ضخمة بتحاول تغطي مسؤوليات متعددة غير مترابطة (نفس مشكلة God Class لكن على مستوى الوحدة).

### ١٥. اعتبارات الأداء

التقسيم لوحدات مالوش تأثير سلبي ملحوظ على الأداء وقت التشغيل في لغات زي Python، لأن عملية الاستيراد (Import) بتحصل مرة واحدة بس عند بدء التشغيل، والفايدة في التنظيم أكبر بكتير من أي تكلفة طفيفة.

### ١٦. الاعتبارات الأمنية

التصميم المعياري بيسمح بتقييد الوصول لوحدات حساسة معينة (زي وحدة التعامل مع بيانات المستخدمين الحساسة) بشكل أوضح، عن طريق تحديد الواجهة العامة اللي الوحدة دي بتعرضها للوحدات التانية بعناية.

## ١٧. اعتبارات قابلية التوسع

التصميم المعياري بيسمح بإضافة وحدات جديدة للمشروع من غير التأثير على الوحدات الموجودة، وده بيدعم التوسع المستمر مع نمو المشروع وزيادة عدد الميزات المطلوبة.

## ١٨. مقارنة مع نهج بديل

البديل هو مشروع بملف واحد أو عدد قليل من الملفات الضخمة من غير تقسيم منطقي حقيقي. الأسلوب ده أسرع في البداية لمشروع صغير جدًا، لكنه بيفشل بسرعة مع النمو، والشركات الكبرى بتستثمر في تصميم معياري واضح من بداية أي مشروع متوقع له عمر طويل.

## ١٩. مثال عملي صغير

```
# Before - everything in one large file
# main.py (500+ lines mixing data loading, model logic, training, evaluation)

# After - modular structure
# project/
#   data/
#     __init__.py
#     loader.py
#   models/
#     __init__.py
#     linear_model.py
#   training/
#     __init__.py
#     trainer.py
#   evaluation/
#     __init__.py
#     metrics.py

# data/__init__.py exposes only what other modules need
from .loader import load_training_data

__all__ = ['load_training_data']
```

## ٢٠. مثال من مشروع Production حقيقي

في منصة ML كبيرة، المشروع بيتقسم لحزم منفصلة زي feature_engineering وmodel_training وmodel_serving، كل واحدة منهم قابلة للتطوير والاختبار بشكل مستقل، وأحيانًا حتى قابلة للتوزيع كمكتبة منفصلة تستخدمها فرق تانية في مشاريع مختلفة.

## ٢١. ملخص الموضوع

**خلاصة Modular Design**

- قسم المشروع لوحدات حسب المسؤولية الوظيفية، مش حسب النوع التقني.
- خلي كل وحدة عالية الترابط داخليًا ومنخفضة الاعتماد على غيرها.
- التصميم المعياري هو الترجمة الفعلية لفكرة الطبقات وفصل الاهتمامات على مستوى المجلدات.

# ٥,٢ Packages — الحزم البرمجية

*الحزمة (Package) هي الآلية اللي بتحول الوحدات (Modules) اللي اتكلمنا عنها في الموضوع اللي فات لكيان قابل للاستيراد والتوزيع وإعادة الاستخدام، سواء داخل نفس المشروع أو عبر مشاريع مختلفة.*

## ١. ليه مفهوم الحزم مهم؟

من غير هيكلة الحزم بشكل صحيح، صعب تستورد كود من وحدة لوحدة تانية بشكل واضح ومنظم، وصعب كمان تشارك كود مفيد (زي أدوات معالجة بيانات شائعة) بين مشاريع مختلفة من غير تكرار الكود ده يدويًا.

في مشاريع ML، الحزم بتسمح بفصل الكود العام القابل لإعادة الاستخدام (زي أدوات معالجة الصور الشائعة) عن كود المشروع المحدد، وده بيسهل الصيانة والتحديث.

## ٢. المشكلة اللي بتحلها الحزم

المشكلة هي الاستيراد الفوضوي بين الملفات من غير هيكلة واضحة، زي استخدام مسارات نسبية معقدة (Relative Imports) بتتكسر بسهولة لما تنقل ملف من مكان لمكان، أو تكرار نفس الكود في أكتر من مشروع لعدم وجود آلية واضحة لمشاركته.

## ٣. إزاي كانت الشركات بتتعامل قبل هيكلة الحزم بشكل احترافي

كان شائع نسخ ولصق الكود المشترك بين مشاريع مختلفة يدويًا، وده كان بيخلي أي تحسين أو إصلاح في الكود المشترك ده يحتاج تحديث يدوي في كل مكان اتنسخ فيه.

## ٤. ليه الطريقة القديمة مش مناسبة

النسخ اليدوي للكود بيخالف مبدأ DRY بشكل مباشر، وبيؤدي لتضارب النسخ المختلفة من نفس الكود بمرور الوقت، حيث كل نسخة بتتطور بشكل منفصل ومحدش متأكد أنهي نسخة هي الصحيحة أو المحدثة.

## ٥. الفكرة الأساسية وراء الحزم

الحزمة هي مجلد بيحتوي على ملف __init__.py (في Python) بيحدد إن المجلد ده حزمة قابلة للاستيراد، مع ملف تعريف (زي pyproject.toml أو setup.py) بيحدد معلومات الحزمة والمكتبات المطلوبة، وده بيسمح بتوزيع الحزمة واستيرادها بشكل موحد.

## ٦. المفاهيم الأساسية

- Package: مجلد بيحتوي على __init__.py، بيمثل وحدة استيراد متكاملة.
- Absolute Imports: الاستيراد باستخدام المسار الكامل للحزمة (زي from project.data.loader import load_data)، وهو الأسلوب المفضل لتجنب مشاكل الاستيراد النسبي.
- Package Distribution: تجهيز الحزمة للتوزيع والتثبيت عن طريق أدوات زي pip، باستخدام ملف pyproject.toml.

## ٧. المكونات

- ملف __init__.py في كل مجلد يمثل حزمة، بيحدد الواجهة العامة المصدرة من الحزمة.
- ملف تعريف الحزمة (pyproject.toml) بيحدد الاسم، الإصدار، والمكتبات المطلوبة (Dependencies).
- هيكل مجلدات واضح يعكس التقسيم المعياري اللي اتفقنا عليه في الموضوع اللي فات.

## ٨. إزاي المبدأ ده بيشتغل داخليًا

لما بايثون بيقابل ملف __init__.py جوه مجلد، بيتعرف على المجلد ده كحزمة قابلة للاستيراد. الاستيراد المطلق (Absolute Import) بيحدد المسار الكامل من جذر المشروع، وده بيخلي الاستيراد متسق بغض النظر عن مكان الملف اللي بينادي على الاستيراد.

## ٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في الشركات الكبرى، فرق الـ ML Platform بتنشئ حزم داخلية (Internal Packages) للأدوات الشائعة (زي أدوات معالجة بيانات موحدة أو أدوات تقييم مقاييس قياسية)، وبتوزعها عن طريق نظام إدارة حزم داخلي (Internal Package Registry)، بحيث أي فريق يقدر يثبت ويستخدم الحزمة دي بنفس سهولة تثبيت أي مكتبة خارجية زي numpy.

### ١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في تنظيم المشروع كحزمة قابلة للتثبيت (pip install -e .)، وفي فصل أدوات المعالجة المشتركة في حزمة داخلية منفصلة يستخدمها أكتر من مشروع، وفي إدارة الاعتماديات (Dependencies) الخاصة بكل حزمة بشكل مستقل.

### ١١. امتى تنظم الكود كحزمة رسمية

- لما الكود متوقع يُستخدم من أكتر من مشروع أو فريق.
- لما المشروع كبير بما يكفي إن الاستيراد المنظم أصبح ضروري لتجنب فوضى المسارات.

### ١٢. امتى ممكن تتجنب الهيكلة الرسمية للحزم

- في سكربت بسيط جدًا أو تجربة واحدة، فيه الاستثمار في هيكلة حزمة كاملة زيادة عن اللزوم.

### ١٣. أفضل الممارسات

**Best Practices**

- استخدم الاستيراد المطلق (Absolute Imports) بدل الاستيراد النسبي المعقد.
- حدد بوضوح في __init__.py إيه اللي الحزمة بتعرضه للاستخدام الخارجي (باستخدام __all__).
- استخدم pyproject.toml لتعريف الحزمة واعتمادياتها بشكل موحد ومعياري.

### ١٤. أشهر الأخطاء الشائعة

**Common Mistakes**

- استخدام استيراد نسبي معقد (زي from ...utils import) بيتكسر بسهولة عند نقل الملفات.
- نسخ نفس الكود المشترك بين مشاريع متعددة بدل تحويله لحزمة داخلية موزعة.
- عدم تحديد الواجهة العامة للحزمة بوضوح، وده بيخلي أي كود خارجي يعتمد على تفاصيل داخلية مش مفروض يعتمد عليها.

### ١٥. اعتبارات الأداء

هيكلة الحزم مالهاش تأثير سلبي على أداء التنفيذ الفعلي، لأن الاستيراد بيحصل مرة واحدة عند بدء التشغيل. لو عدد الحزم كبير جدًا ومتشابك، وقت بدء التشغيل (Startup Time) ممكن يزيد شوية، لكن ده نادرًا ما يكون مشكلة حقيقية في مشاريع ML.

### ١٦. الاعتبارات الأمنية

لو بتستخدم حزم من مصادر خارجية، لازم التحقق من مصدرها وموثوقيتها، خصوصًا في نظام إدارة الحزم الداخلي للشركة، لتجنب استيراد كود ضار أو حزم مزيفة (Typosquatting) بأسماء مشابهة لحزم موثوقة.

### ١٧. اعتبارات قابلية التوسع

الحزم الداخلية بتسمح بمشاركة الكود المشترك بين عدد كبير من الفرق والمشاريع بكفاءة، وده بيدعم التوسع على مستوى المؤسسة كلها، مش بس على مستوى مشروع واحد.

## ١٨. مقارنة مع نهج بديل

البديل هو نسخ الكود المشترك يدويًا بين المشاريع المختلفة. الأسلوب ده أسرع لمرة واحدة، لكنه بيؤدي لتضارب النسخ وصعوبة الصيانة مع الوقت. الشركات الكبرى بتستثمر في بنية تحتية لإدارة الحزم الداخلية لتجنب المشكلة دي من الأساس.

## ١٩. مثال عملي صغير

```
# Project structure as an installable package
# my_ml_project/
#   pyproject.toml
#   my_ml_project/
#     __init__.py
#     data/
#       __init__.py
#       loader.py

# pyproject.toml
# [project]
# name = "my_ml_project"
# version = "0.1.0"
# dependencies = ["pandas", "scikit-learn"]

# Absolute import from anywhere in the project
from my_ml_project.data.loader import load_training_data
```

## ٢٠. مثال من مشروع Production حقيقي

في شركة كبيرة، حزمة داخلية اسمها company_ml_utils بتحتوي على أدوات معالجة بيانات ومقاييس تقييم موحدة، وبتتثبت زي أي مكتبة خارجية (pip install company-ml-utils) في عشرات المشاريع المختلفة داخل الشركة، وأي تحسين في الحزمة دي بيستفيد منه كل المشاريع دي فورًا بمجرد تحديث الإصدار.

## ٢١. ملخص الموضوع

**خلاصة Packages**

- الحزمة بتحول الوحدات لكيان قابل للاستيراد والتوزيع بشكل منظم.
- استخدم الاستيراد المطلق وحدد الواجهة العامة للحزمة بوضوح.
- الحزم الداخلية بتمنع تكرار الكود وتسهل مشاركته بين مشاريع وفرق متعددة.

# ٥,٣ Configuration — إدارة الإعدادات

*الإعدادات (Configuration) هي القيم اللي بتحدد سلوك النظام من غير الحاجة لتعديل الكود نفسه، زي مسار البيانات، أو Hyperparameters النموذج، أو عنوان قاعدة البيانات. إدارتها بشكل صحيح جزء أساسي من فصل الاهتمامات اللي اتعلمناه في الفصل السابق.*

## ١. ليه إدارة الإعدادات بشكل صحيح مهمة؟

لو القيم دي متكتوبة مباشرة (Hardcoded) جوه الكود، أي تغيير فيها بيحتاج تعديل الكود نفسه وإعادة نشره، حتى لو التغيير بسيط زي تجربة Learning Rate مختلف. إدارة الإعدادات بشكل منفصل بتسمح بتغيير سلوك النظام من غير لمس الكود الأساسي.

في مشاريع ML، ده مهم جدًا لأن التجريب (Experimentation) بيعتمد بشكل أساسي على تجربة قيم Hyperparameters مختلفة بسرعة، من غير الحاجة لتعديل الكود في كل مرة.

## ٢. المشكلة اللي بتحلها إدارة الإعدادات الصحيحة

المشكلة هي القيم المكتوبة مباشرة جوه الكود (Hardcoded Values)، زي learning_rate = 0.01 مكتوبة جوه دالة التدريب مباشرة. أي تجربة بقيمة مختلفة بتحتاج تعديل الكود نفسه، وده بيزود احتمال الأخطاء ويصعب تتبع أنهي قيمة اتجربت في أنهي تجربة.

## ٣. إزاي كانت الشركات بتتعامل قبل إدارة الإعدادات بشكل منظم

كان شائع كتابة القيم المهمة مباشرة جوه الكود، وأي تجربة بقيم مختلفة كانت بتتطلب تعديل الكود يدويًا، وأحيانًا الاحتفاظ بنسخ متعددة من نفس السكربت بقيم مختلفة لكل تجربة.

## ٤. ليه الطريقة القديمة مش مناسبة

القيم المكتوبة مباشرة بتخلي تتبع التجارب المختلفة فوضويًا، وبتزود احتمال الأخطاء البشرية (زي نسيان تحديث قيمة معينة)، وبتصعب مقارنة نتائج التجارب المختلفة بشكل منظم ومنهجي.

## ٥. الفكرة الأساسية وراء إدارة الإعدادات

كل القيم القابلة للتغيير بتتحط في ملفات إعدادات منفصلة عن الكود (زي YAML أو JSON)، والكود بيقرأ القيم دي وقت التشغيل بدل ما تكون مكتوبة بداخله مباشرة، وده بيسمح بتغيير السلوك من غير تعديل الكود.

## ٦. المفاهيم الأساسية

- Configuration File: ملف منفصل (YAML، JSON، TOML) بيحتوي على القيم القابلة للتغيير.
- Configuration Object: كلاس أو هيكل بيانات بيمثل الإعدادات بشكل منظم بعد قراءتها من الملف، بدل التعامل مع Dictionary عشوائي.
- Environment-Specific Configuration: إعدادات مختلفة حسب البيئة (تطوير، اختبار، إنتاج)، بحيث نفس الكود يشتغل بسلوك مختلف حسب البيئة من غير تعديل.

## ٧. المكونات

- ملف إعدادات لكل بيئة أو تجربة (زي config/development.yaml وconfig/production.yaml).
- كلاس Configuration بيحمل القيم بشكل منظم وموثق بعد قراءتها من الملف.
- آلية تحميل الإعدادات المناسبة حسب البيئة الحالية وقت بدء التشغيل.

## ٨. إزاي المبدأ ده بيشتغل داخليًا

عند بدء تشغيل النظام، بيتم تحديد البيئة الحالية (غالبًا عن طريق Environment Variable)، وبيتم تحميل ملف الإعدادات المناسب لها، وتحويله لكائن Configuration منظم. باقي الكود بيتعامل مع الكائن ده بدل القيم المباشرة، وده بيطبق نفس فكرة Dependency Injection اللي اتعلمناها في فصل SOLID: الكود بيعتمد على واجهة الإعدادات، مش على القيم الفعلية المحددة.

## ٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في منصات ML الكبرى، كل تجربة (Experiment) بيكون ليها ملف إعدادات خاص بيها بيحدد كل شيء: النموذج المستخدم، الـ Hyperparameters، مصدر البيانات. ده بيسمح بتتبع دقيق لكل تجربة (Experiment Tracking) وإعادة إنتاج نفس النتيجة بدقة لو احتجت تكرر التجربة لاحقًا.

## ١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في ملفات إعدادات التجارب المختلفة، وفي إعدادات الاتصال بمصادر البيانات لكل بيئة، وفي إعدادات مستوى الـ Logging المختلف بين بيئة التطوير (تفاصيل كتيرة) وبيئة الإنتاج (تفاصيل أقل لتوفير الأداء).

## ١١. امتى تفصل الإعدادات عن الكود بصرامة

- لما القيمة متوقع تتغير بين تجارب أو بيئات مختلفة.
- لما القيمة حساسة (زي مفاتيح API) ومش المفروض تتكتب مباشرة جوه الكود المصدري.

## ١٢. امتى ممكن تسيب القيمة جوه الكود

- لما القيمة ثابتة بطبيعتها ومش متوقع تتغير أبدًا (زي قيمة رياضية ثابتة معروفة).

## ١٣. أفضل الممارسات

**Best Practices**

- افصل كل القيم القابلة للتغيير في ملفات إعدادات منفصلة عن الكود.
- استخدم كائن Configuration منظم بدل التعامل المباشر مع Dictionary عشوائي.
- احتفظ بملف إعدادات منفصل لكل بيئة (تطوير، اختبار، إنتاج) بدل خلطهم في ملف واحد بشروط معقدة.

## ١٤. أشهر الأخطاء الشائعة

**Common Mistakes**

- كتابة قيم مهمة مباشرة جوه الكود (Hardcoded Values) بدل ملفات إعدادات منفصلة.
- خلط الإعدادات الحساسة (زي مفاتيح API) مع الإعدادات العادية في نفس الملف من غير حماية إضافية.
- عدم توثيق القيم الافتراضية والمعنى الحقيقي لكل إعداد، وده بيصعب على مهندس جديد يفهم تأثير كل قيمة.

## ١٥. اعتبارات الأداء

قراءة الإعدادات من ملف خارجي بتحصل مرة واحدة بس عند بدء التشغيل، فالتأثير على الأداء مهمل تمامًا مقارنة بفايدة المرونة في تغيير السلوك من غير تعديل الكود.

## ١٦. الاعتبارات الأمنية

القيم الحساسة (زي مفاتيح API أو كلمات مرور قواعد البيانات) ميفروضش تُكتب مباشرة في ملفات الإعدادات العادية اللي بتتحفظ في نظام التحكم بالإصدارات (Git)، ولازم تُدار عن طريق آلية منفصلة أكتر أمانًا زي Environment Variables أو أنظمة إدارة الأسرار (Secrets Management)، وده الموضوع اللي هنشرحه بالتفصيل في الموضوع الجاي.

## ١٧. اعتبارات قابلية التوسع

إدارة الإعدادات بشكل منظم بتسمح بإضافة بيئات جديدة (زي بيئة اختبار أداء منفصلة) من غير التأثير على الكود الأساسي، وده بيدعم التوسع لدعم سيناريوهات تشغيل متعددة مع نمو المشروع.

## ١٨. مقارنة مع نهج بديل

البديل هو كتابة القيم مباشرة جوه الكود وتعديلها يدويًا لكل تجربة أو بيئة. الأسلوب ده أسرع لتجربة واحدة سريعة، لكنه بيفشل بسرعة مع تعدد التجارب والبيئات، والشركات الكبرى بتفرض فصل الإعدادات عن الكود كمعيار أساسي لأي مشروع Production.

## ١٩. مثال عملي صغير

```
# config/training.yaml
# model_type: xgboost
# learning_rate: 0.01
# batch_size: 32
# data_path: /data/training.csv

import yaml
from dataclasses import dataclass

@dataclass
class TrainingConfig:
    model_type: str
    learning_rate: float
    batch_size: int
    data_path: str

def load_config(path: str) -> TrainingConfig:
    with open(path) as f:
        raw = yaml.safe_load(f)
    return TrainingConfig(**raw)

config = load_config('config/training.yaml')
model = ModelFactory.create(config.model_type)
```

## ٢٠. مثال من مشروع Production حقيقي

في نظام تجريب ML حقيقي، كل تجربة بيتم تشغيلها بملف إعدادات منفصل يوضح كل التفاصيل (النموذج، الـ Hyperparameters، مصدر البيانات)، وبيتم حفظ الملف ده مع نتائج التجربة، بحيث أي حد يقدر يرجع لاحقًا ويعرف بالظبط إزاي اتعملت التجربة دي ويقدر يعيد إنتاجها بدقة.

## ٢١. ملخص الموضوع

**خلاصة Configuration**

- افصل القيم القابلة للتغيير عن الكود في ملفات إعدادات منفصلة.
- استخدم كائن Configuration منظم بدل Dictionary عشوائي.

- القيم الحساسة تحتاج آلية منفصلة أكتر أمانًا، مش ملفات إعدادات عادية.

# ٥,٤ Environment Variables — متغيرات البيئة

*متغيرات البيئة (Environment Variables) هي آلية لتمرير قيم للبرنامج من خارج الكود ومن خارج ملفات المشروع نفسها، وبيتم تعريفها على مستوى نظام التشغيل أو بيئة التشغيل (Runtime Environment) اللي البرنامج شغال فيها.*

## ١. ليه متغيرات البيئة مهمة؟

بعض القيم حساسة جدًا (زي مفاتيح API، كلمات مرور قواعد البيانات) ومش المفروض تتكتب في أي ملف بيتحفظ في نظام التحكم بالإصدارات (Git)، حتى لو كان ملف إعدادات منفصل. متغيرات البيئة بتوفر آلية لتمرير القيم دي وقت التشغيل من غير ما تكون موجودة في أي ملف مصدري.

في مشاريع ML، ده بيظهر في تمرير مفاتيح الوصول لخدمات التخزين السحابي، أو بيانات الاتصال بقواعد بيانات الإنتاج، من غير ما تكون مكتوبة في أي مكان في الكود أو ملفات الإعدادات العادية.

## ٢. المشكلة اللي بتحلها متغيرات البيئة

المشكلة هي القيم الحساسة اللي بتتكتب مباشرة في الكود أو حتى في ملفات الإعدادات العادية، وده بيعرضها لخطر التسريب لو الملف ده اتحفظ بالغلط في نظام التحكم بالإصدارات العام أو اتشارك مع طرف غير مصرح له.

## ٣. إزاي كانت الشركات بتتعامل قبل استخدام متغيرات البيئة بشكل صحيح

كان شائع كتابة مفاتيح API وكلمات المرور مباشرة جوه الكود أو ملفات الإعدادات، وأحيانًا كانت بتتسرب بالغلط لما الملفات دي كانت بتتحفظ في مستودعات Git عامة، وده تسبب في حوادث أمنية حقيقية معروفة في الصناعة.

## ٤. ليه الطريقة القديمة مش مناسبة

أي ملف بيتحفظ في نظام التحكم بالإصدارات معرض لخطر المشاركة غير المقصودة، وحتى لو اتحذف لاحقًا، القيمة الحساسة بتفضل موجودة في تاريخ الإصدارات (Git History) إلا لو اتعمل تنظيف متخصص للتاريخ ده، وهي عملية معقدة ومش مضمونة النجاح الكامل.

## ٥. الفكرة الأساسية وراء متغيرات البيئة

القيم الحساسة بتتحط في بيئة التشغيل نفسها (Operating System أو Container Environment)، مش في أي ملف بيتحفظ مع الكود، والكود بيقرأ القيم دي وقت التشغيل عن طريق واجهة نظام التشغيل، بحيث القيمة الفعلية مش موجودة في أي مكان يتحفظ أو يتشارك مع الكود المصدري.

## ٦. المفاهيم الأساسية

- Environment Variable: قيمة معرفة على مستوى نظام التشغيل أو بيئة التشغيل، بتتقرأ من البرنامج وقت التشغيل.
- env File. (للتطوير المحلي فقط): ملف بيحدد متغيرات البيئة لبيئة التطوير المحلية، ولازم يكون مستثنى من نظام التحكم بالإصدارات (.gitignore).
- Secrets Management System: نظام متخصص (زي AWS Secrets Manager أو HashiCorp Vault) لإدارة القيم الحساسة بشكل أكثر أمانًا من متغيرات البيئة العادية في بيئات الإنتاج الكبيرة.

## ٧. المكونات

- تعريف متغيرات البيئة المطلوبة في بيئة التشغيل (محليًا، أو في نظام الـ CI/CD، أو في بيئة الإنتاج).
- كود لقراءة القيم دي وقت التشغيل (زي os.environ في Python) والتحقق من وجودها.
- ملف .env.example (بدون قيم فعلية حساسة) بيوثق أسماء المتغيرات المطلوبة لأي مهندس جديد.

## ٨. إزاي المبدأ ده بيشتغل داخليًا

عند بدء تشغيل البرنامج، بيقدر يقرأ القيم المعرفة في بيئة التشغيل عن طريق واجهة نظام التشغيل، من غير ما القيم دي تكون مكتوبة في أي مكان في الكود المصدري نفسه. القيم دي بتتحط في بيئة التشغيل بطرق مختلفة حسب السياق: يدويًا في بيئة التطوير، أو عن طريق نظام إدارة الأسرار في بيئة الإنتاج.

### ٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في الشركات الكبرى، بيئات الإنتاج بتستخدم أنظمة إدارة أسرار متخصصة (Secrets Management Systems) بدل متغيرات البيئة العادية، وده بيوفر ميزات إضافية زي التدوير الدوري التلقائي للمفاتيح (Automatic Rotation) وتسجيل دقيق لمن قام بالوصول لأي قيمة حساسة ومتى.

### ١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في تمرير مفاتيح الوصول لخدمات التخزين السحابي (زي AWS أو GCP)، بيانات الاتصال بقواعد بيانات الإنتاج، مفاتيح API لخدمات خارجية (زي خدمات المراقبة أو الإشعارات)، وأي قيمة حساسة تانية مش المفروض تكون مرئية في الكود المصدري.

### ١١. امتى تستخدم متغيرات البيئة

- لأي قيمة حساسة (مفاتيح API، كلمات مرور، بيانات اتصال).
- لأي قيمة متوقع تختلف بين البيئات المختلفة بشكل حساس (زي عنوان قاعدة بيانات الإنتاج مقابل التطوير).

### ١٢. امتى ملفات الإعدادات العادية كافية

- للقيم غير الحساسة اللي مش مشكلة لو ظهرت في نظام التحكم بالإصدارات، زي Hyperparameters النموذج أو إعدادات غير سرية.

### ١٣. أفضل الممارسات

**Best Practices**

- خلي كل القيم الحساسة تُدار عن طريق متغيرات البيئة أو نظام إدارة أسرار متخصص، مش ملفات إعدادات عادية.
- استثنِ ملفات .env من نظام التحكم بالإصدارات (.gitignore) بشكل صريح.
- وفر ملف .env.example موثق بأسماء المتغيرات المطلوبة (من غير قيم فعلية) لمساعدة أي مهندس جديد.

### ١٤. أشهر الأخطاء الشائعة

**Common Mistakes**

- حفظ ملف .env يحتوي على قيم حقيقية في نظام التحكم بالإصدارات بالغلط.
- كتابة مفاتيح API مباشرة جوه الكود المصدري للاختبار السريع، ونسيان إزالتها قبل الدفع (Commit) للمستودع.

عدم التحقق من وجود متغيرات البيئة المطلوبة عند بدء التشغيل، وده بيؤدي لأخطاء غامضة لاحقًا في منتصف التنفيذ بدل رسالة خطأ واضحة من البداية.

### ١٥. اعتبارات الأداء

قراءة متغيرات البيئة بتحصل مرة واحدة بس عند بدء التشغيل، فالتأثير على الأداء وقت التشغيل الفعلي مهمل تمامًا.

### ١٦. الاعتبارات الأمنية

متغيرات البيئة أفضل بكتير من كتابة القيم مباشرة في الكود، لكنها مش الحل الأكثر أمانًا في بيئات إنتاج كبيرة وحساسة، حيث أنظمة إدارة الأسرار المتخصصة بتوفر حماية إضافية زي التشفير أثناء التخزين، والتدوير التلقائي للمفاتيح، وتسجيل دقيق لأي وصول للقيم الحساسة.

## ١٧. اعتبارات قابلية التوسع

استخدام متغيرات البيئة بيسمح بتشغيل نفس الكود في بيئات متعددة (تطوير، اختبار، إنتاج، وحتى عملاء مختلفين في حالة الأنظمة متعددة المستأجرين Multi-Tenant) من غير أي تعديل في الكود نفسه، بس بتغيير القيم المعرفة في كل بيئة.

## ١٨. مقارنة مع نهج بديل

البديل هو تخزين القيم الحساسة في ملفات إعدادات عادية مع محاولة حمايتها بطرق يدوية (زي تشفير الملف يدويًا). الأسلوب ده أضعف بكتير وأكتر عرضة للخطأ البشري من الاعتماد على متغيرات البيئة أو أنظمة إدارة الأسرار المتخصصة، اللي مصممة خصيصًا للتعامل مع هذا النوع من المخاطر.

## ١٩. مثال عملي صغير

```
import os

def get_required_env(name: str) -> str:
    value = os.environ.get(name)
    if value is None:
        raise RuntimeError(f'Missing required environment variable: {name}')
    return value

database_url = get_required_env('DATABASE_URL')
api_key = get_required_env('CLOUD_STORAGE_API_KEY')

# .env.example (committed to version control, no real values)
# DATABASE_URL=
# CLOUD_STORAGE_API_KEY=

# .env (real values, excluded via .gitignore)
# DATABASE_URL=postgres://user:pass@prod-db:5432/mldb
# CLOUD_STORAGE_API_KEY=sk-real-key-here
```

## ٢٠. مثال من مشروع Production حقيقي

في نظام ML بيشتغل على منصة سحابية، مفتاح الوصول لخدمة التخزين السحابي وبيانات الاتصال بقاعدة بيانات الإنتاج بيتم حقنهم في بيئة التشغيل (Container Environment) عن طريق نظام إدارة أسرار مركزي، من غير ما تظهر القيم دي في أي مكان في الكود المصدري أو في سجلات النظام (Logs).

## ٢١. ملخص الموضوع

**خلاصة Environment Variables**

- القيم الحساسة لازم تُدار عن طريق متغيرات البيئة، مش ملفات إعدادات عادية أو الكود المصدري.
- استثنِ ملفات .env من نظام التحكم بالإصدارات ووفر ملف .env.example موثق.
- في بيئات الإنتاج الكبيرة، أنظمة إدارة الأسرار المتخصصة أفضل من متغيرات البيئة العادية.

## مشروع عملي على الفصل: إعادة تنظيم مشروع ML من سكربت واحد لبنية احترافية كاملة

المشروع ده هدفه إنك تاخد سكربت ML واحد فوضوي وتحوله لمشروع منظم بالكامل، مطبقًا التصميم المعياري والحزم وإدارة الإعدادات ومتغيرات البيئة مع بعض.

السيناريو: عندك ملف train.py واحد بيحتوي على كل حاجة: تحميل بيانات، تدريب نموذج، مفاتيح API مكتوبة مباشرة، وقيم Hyperparameters ثابتة جوه الكود.

### خطوات التنفيذ

- طبق Modular Design: قسم الملف الواحد لوحدات منفصلة (data، models، training، evaluation).
- حول المشروع لحزمة (Package): أضف __init__.py وpyproject.toml، واستخدم استيراد مطلق بين الوحدات.
- طبق Configuration: انقل كل الـ Hyperparameters والقيم القابلة للتغيير لملف config/training.yaml منفصل.
- طبق Environment Variables: انقل مفتاح الـ API وبيانات الاتصال الحساسة لمتغيرات بيئة، وأضف ملف env.example. موثق.
- تأكد إن الملف .env الحقيقي مستثنى من .gitignore، وإن ملف الإعدادات العادي خالي تمامًا من أي قيمة حساسة.

## تحديات شائعة عند التطبيق

- الخلط بين Configuration وEnvironment Variables: القاعدة العملية هي إن أي قيمة حساسة (مفاتيح، كلمات مرور) تروح لمتغيرات البيئة، وأي قيمة غير حساسة لكن قابلة للتغيير (زي Hyperparameters) تروح لملف إعدادات عادي.
- المبالغة في تقسيم المشروع لوحدات صغيرة جدًا في مرحلة مبكرة جدًا من المشروع، وده بيزود تعقيد التنقل بين الملفات من غير فايدة حقيقية بعد. القاعدة العملية: ابدأ ببنية بسيطة معقولة، وقسم أكتر لما الحاجة الفعلية تظهر.
- نسيان توثيق متغيرات البيئة المطلوبة، وده بيخلي أي مهندس جديد يضطر يقرأ الكود كله عشان يعرف إيه المتغيرات اللي محتاج يجهزها قبل ما يقدر يشغل المشروع.

## إزاي الفصل ده بيتستخدم داخل مشروع Machine Learning حقيقي

التنظيم اللي اتعلمناه في الفصل ده هو أول حاجة بيشوفها أي مهندس جديد لما ينضم لفريق ML: هيكل المجلدات (Modular Design وPackages) بيديله خريطة واضحة للمشروع، وملفات الإعدادات (Configuration) بتوضحله إزاي يشغل تجربة جديدة، ومتغيرات البيئة بتوضحله إيه القيم الحساسة اللي محتاج يجهزها بنفسه محليًا.

في الشركات الكبرى، أي مشروع ML جديد بيبدأ من قالب موحد (Project Template) بيطبق كل المبادئ دي من اليوم الأول، عشان يضمن اتساق التنظيم عبر كل المشاريع المختلفة داخل المؤسسة، وده بيسهل بشكل كبير انتقال المهندسين بين فرق ومشاريع مختلفة.

## ملخص الفصل الخامس وربطه بالفصل القادم

في الفصل ده اتعلمت إزاي تترجم مبادئ Clean Architecture لتنظيم فعلي وملموس على مستوى نظام الملفات: التصميم المعياري (Modular Design) لتقسيم المشروع لوحدات واضحة المسؤولية، الحزم (Packages) لجعل الوحدات دي قابلة للاستيراد والتوزيع، إدارة الإعدادات (Configuration) لفصل القيم القابلة للتغيير عن الكود، ومتغيرات البيئة (Environment Variables) للتعامل الآمن مع القيم الحساسة.

الفصل الجاي (Error Handling) هيبني مباشرة على التنظيم ده. هنشرح إزاي نتعامل مع الأخطاء (Exceptions) بشكل احترافي، وإزاي نتحقق من صحة المدخلات (Validation)، وإزاي نسجل الأحداث (Logging) بشكل مفيد - وكل ده هيعتمد على وجود بنية واضحة تحدد فين بالظبط المفروض يحصل كل نوع من التعامل مع الأخطاء، ومستوى التفاصيل المطلوب في كل بيئة حسب إعداداتها.